Proof notes: what each result actually says and why it's true

Hoang Nguyen

Abstract

Most of the theorems below sound like they should just be true. "Merging two regions doesn't create a duplicate id." "A field write somewhere active can't disturb a suspended region." "Once a region is closed, the only way in is through the front door." Intuition gives you the headline for free. It doesn't give you the proof: in each case below, intuition either quietly assumes a fact that first has to be derived from the model's invariants, or is wrong as stated until it's qualified in a way you wouldn't notice without trying to break it. The point of this document is to make that gap visible, for someone who trusts the model's design (`writeup/main.tex`, `lean/gc/README.md`) but hasn't looked at how any of these results actually got proved.

Each result below is given as a claim, what intuition already covers, where the real work sits, and how the proof gets there, so that each one can be read and retold on its own. A reference catalog of the reusable lemmas and known pitfalls follows at the end, meant for looking things up once you already know which proof you're asking about.

Scope: `Gc/Model/` and `Gc/Reachability/Reachable/`. `Gc/Reachability/Referencable/` is left out on purpose; it's deprecated and superseded by `Reachable/` (see `CLAUDE.md`).

Contents

1 Preliminaries

1.1 The model and its rules

The configuration $\langle S, H\rangle$ (stack of frames, heap of regions) and the nine mutation operations' operational-semantics rules are given in full in `writeup/main.tex`, in its "Model" and "Operational Semantics" sections.[1] We reuse that notation throughout: $\mathsf{Rld}(rid)/\mathsf{Old}(oid)$ for references, $a.loc(\langle S, H\rangle)$ for where a reference resolves ($\mathsf{Stk}(fid)$, $\mathsf{Rgn}(rid)$, or undefined), $a.objAt(\langle S, H\rangle)$ for the object an Old reference currently names.

1.2 The ten invariants of *ValidConfig*

`main.tex`'s "Invariants" section defines all ten (*L1*, *L2*, *H1*, *H2*, *H3*, *S1*, *S2*, *S3*, *HS1*, *HS2*) and bundles them into *ValidConfig*. We won't restate the formulas here. The table below is just a reminder of what each one forbids, since that's the fact every proof below actually reaches for.

Invariant	What it rules out
L1	the same object id allocated twice (stack or heap, anywhere)
L2	a frame executing inside a region that is actually *Closed*
H1	a region whose bridge object isn't actually a member of its own *objMap*
H2	two live $\mathsf{Rld}(rid)$ references to the same region
H3	a region's contents holding a reference that resolves outside that region
S1	two stack frames both claiming the same region
S2	a frame referencing a stack object owned by a later frame
S3	a frame referencing a region owned only by a later frame
HS1	a dangling Old (points at an id never/no-longer allocated)
HS2	a dangling Rld (points at a region no longer in the heap)

Remark 1.1. *HS1*/*HS2* don't follow from the rest. *H2*, *H3*, *S2*, *S3* all presuppose a reference that already resolves somewhere; none of them says anything about one that resolves to nothing. Without *HS1*/*HS2*, a stale reference could sit inert and later collide, by coincidence, with a freshly allocated id, and nothing above would have caught the staleness in between. (`main.tex` makes the same point. It surfaced while proving *S2*/*S3* preservation for the three `make*` operations, whose `freshObjectId`/`freshRegionId` only look at ids currently used as keys.)

Four corollaries proved directly from the invariants (`Validity.lean`) get cited constantly by everything downstream, in both layers:

- `oid_in_stack_implies_not_in_heap` / `oid_in_heap_implies_not_in_stack`: from *L1*, an id found by one search can't also be found by the other.
- `oid_loc_rgn_iff_in_heap`, `oid_loc_stk_iff_in_stack`, `rid_loc_rgn_iff_in_heap`: these rewrite the opaque computation $\mathsf{Old}(oid).loc(\langle S, H\rangle) = \mathsf{Rgn}(rid)$ into the far more usable $\exists region.\ H(rid) = region \wedge oid \in region.objMap$. Anywhere that needs to reason about where an id lives goes through one of these three, not through the raw definition of $.loc(\cdot)$.

[1] This document compiles standalone, so cross-file `\ref`s to `main.tex`'s labels aren't used; section names are given in prose instead.

1.3 *RuntimeConfig.start* satisfies *ValidConfig*

One root region (Open, bridge object 0) and one root frame executing in it. Each of the ten invariants reduces, by unfolding, to a fact about a one- or two-element list, closed by `decide/simp/List.nodup_singleton`. Nothing interesting happens here; the file exists to anchor the induction below.

1.4 The preservation proof pattern

Every `Gc/Model/Mutation/<Op>.lean` defines the operation as an *Option RuntimeConfig*-valued `do`-block, each bind a precondition that can fail, matching the premises of the corresponding inference rule in `main.tex`. It's paired with an `<op>_cases` lemma that unpacks a successful run into an explicit disjunction of fully-instantiated hypotheses, one disjunct per rule variant. This is mechanical, but it's also the real specification of the operation: a mistake here, a missing disjunct or a wrong side condition, wouldn't show up as a Lean error in the case lemma's own proof (which just replays the `do`-block's control flow); it would only show up later, as an unprovable or vacuously provable downstream theorem. Every proof from here on, in both layers, destructs a hypothesis $op\ cfg = \texttt{some}\ cfg'$ via `<op>_cases` exactly once, then argues from the concrete named values it produces.

`Preservation/<Op>.lean` then proves ten lemmas `<op>_L1`, …, `<op>_HS2` and combines them into `<op>_valid` : $\mathit{ValidConfig}(cfg) \to op\ cfg = \texttt{some}\ cfg' \to \mathit{ValidConfig}(cfg')$. `Preservation.lean` aggregates all nine into *AllPreserve*(*ValidConfig*) by case-splitting on the reified *Stmt* type (`Mutation/Stmt.lean`), so "every one of the nine operations preserves *ValidConfig*" is proved once, generically, rather than left as nine separate theorems a caller has to know to look up.

2 How the proofs actually work

Five walkthroughs, chosen to cover both layers and every distinct proof technique in the codebase. §2.1 only needs §1.2 above. §2.3–2.6 concern `Gc/Reachability/Reachable/`, so §2.2 first recaps just enough of that layer's definitions to make them readable on their own (all stated in full in `main.tex`).

2.1 `merge` keeps the configuration valid

The claim. After `merge x` folds a Closed region `rid'` into the current Open region `rid` (combining their *objMap*s, deleting `rid'` from the heap, rebinding `x` to `rid'`'s old bridge object), the result is still a *ValidConfig*: in particular it still has no duplicate object ids (*L1*), and every region's contents still resolve back into that same region (*H3*).

The intuitive story. "We're just pouring one region's objects into another. Nothing is created or destroyed, so of course ids stay unique, and anything that already pointed home still points home." That's the right story, and as far as a reader is concerned it's the whole content of the theorem. But "nothing is created or destroyed" is a claim about the operation's semantics, and the proof has to earn it from `merge`'s actual precondition, not from the English description of what it's supposed to do.

Where the two facts above actually come from.

- *Disjointness has to be derived, it isn't given.* Nothing in `merge`'s `do`-notation states that the two regions' object-id sets are disjoint. If they weren't, the union could identify two previously-different objects (breaking *L1* in the other direction), or, since *AList.union* is implemented via *kunion*/*kerase* and erases duplicate keys from one side, it could silently

drop an object nobody asked to delete. The only fact the precondition actually gives is that one region is Open and the other Closed. Since those statuses are mutually exclusive, and *L1* says no id lives in two heap entries at once, that alone rules out a shared id between the two regions.

- *H3 isn't a relabelling, it's a fresh search.* An object that used to live in the now-deleted region `rid'` has to be shown to resolve, post-merge, into `rid`. But $.loc(\cdot)$ has no memory: it runs a brand-new search over whatever heap exists right now. So the proof has to reconstruct, from scratch, that searching the post-merge heap for this object's id lands on `rid` and nowhere else, which again needs disjointness, this time to rule out some other heap entry also matching.

The argument itself.

1. `merge_corollary_rid_ne_regionId`: `rid` and `rid'` are provably distinct heap entries, since Open $\neq$ Closed.
2. `merge_corollary_region_unique` (an *L1* corollary): no object id can appear in two different heap entries' object maps, so the two regions share no id.
3. `merge_corollary_union_append`: on disjoint keys, $s_1 \cup s_2$'s `.entries` really is just $s_1.entries +\!\!+ s_2.entries$. This is what licenses treating the merged region's contents as "*region*'s own entries, then *region′*'s, nothing else," instead of trusting *AList.union*'s general, lossier behaviour.
4. `merge_corollary_heap_entries_perm`: the whole post-merge heap's entry list is a permutation of the merged pair plus everything untouched, built by pulling `rid` and `rid'` out of the original heap one at a time (two nested `List.exists_of_kerase` extractions).
5. *L1* then comes down to permutation-invariance: `List.count`/`Nodup` don't care about reordering, so knowing the merged heap's object-id list is a permutation of the original is enough.
6. *H3*: `merge_corollary_loc_of_mem` shows an affected object still resolves into `rid` post-merge, by showing the stack side of $.loc(\cdot)$'s search is untouched (`merge` only rewrites the current frame's *varMap*, never its objMap), and that on the heap side the uniqueness from steps 2 and 4 pins `rid`'s merged entry down as the only heap entry whose objMap could contain this id.

So the hard part of `merge` was never the union itself. It was showing, from the operation's own precondition, that the two regions being combined could never have shared an id, and then re-deriving from scratch where every affected object resolves under the new heap, since $.loc(\cdot)$ has no memory of where anything used to be.

2.2 Reachability, recapped

$a \to b$ (*ReachableStep*) is one hop of a reference chain. The only content beyond following an object's own stored references: $\mathsf{RId}(rid) \to b$ is possible only when $H(rid)$ is Closed, and it forces $b = \mathsf{OId}(H(rid).bridgeObjectId)$. A Closed region's RId has exactly one outgoing step, into its own bridge object; an Open region's RId has none at all (by *L2*: if it could step, it would have to be Closed, contradicting an on-stack frame owning it). A chain may step into a suspended region through this one door, but never out. That asymmetry is the whole reason this layer exists, and it's what makes §2.4 non-trivial.

$RegionReachable(rid, \cdot)$/$FrameReachable(fid, \cdot)$ are the inductive closures of $\to$ from a region's bridge object, or from a frame's roots (a variable's value, or that frame's own region's bridge object, i.e. *FrameRoot*); *StackReachable* means *FrameReachable* from some on-stack frame. `main.tex`'s $\xrightarrow{*}$-reformulation theorems restate all of these as "there is a root, and a `Relation.ReflTransGen`($\to$) chain from it," and it's this restated form, not the raw inductive definition, that every proof below actually inducts on. `ReflTransGen` comes with two mathlib induction principles: ordinary

constructor induction fixes the chain's first element and grows the last one step at a time, good for tracing forward from a known root, while `head_induction_on` fixes the last element and peels off the first step, good for chasing an escaping chain hop by hop from a known start. Picking the wrong one doesn't produce a Lean error. It produces a stuck goal whose motive quietly fails to generalize the variable the argument needs.

2.3 Reaching backward in time

The claim (`region_container_confined`, `stack_container_confined`). If you can trace a reference chain, starting from some frame at index fid, back to an object sitting inside a particular (Open) region, or a particular stack frame, then whichever frame actually owns that container has index no later than fid.

The intuitive story. You can't reach an earlier or more-nested frame's stuff from a frame that comes later unless something already linked the two, and that sounds close to a restatement of the stack discipline: *S2*/*S3* already say more or less exactly this.

What they actually say, and what's missing. *S2*/*S3* are one-hop facts. They bound a single step of a chain. The claim above is about a chain of arbitrary, unknown length, handed to the proof only as an opaque `ReflTransGen` witness. Getting from "true for one hop" to "true for the whole chain" takes an induction that walks the chain and re-establishes the bound at every hop, with a real case split for the moment a chain crosses from inside the region onto the stack, since that's a different kind of hop from staying inside.

The argument.

1. Restate the *FrameReachable* witness in its *ReflTransGen* form (§2.2); the custom inductive relation's own recursion runs the wrong direction for this.
2. Generalize over the chain's end (the reachable object) and induct forward from the fixed start. At each step, either the chain is still inside the region, in which case `predecessor_of_region_object` (an *H3*-driven fact: in an Open region, anything one hop from an in-region object is itself in that region or has escaped to the stack, never a bare RId or a different region) lets you recurse, or the chain has just escaped to the stack, in which case the one-hop bound (`stack_step_region_index_le`, i.e. *S3* restated at the single-hop level) gives the numeric bound directly and the induction stops.
3. The two ways a chain can start, a variable slot or a frame's own bridge object, are each closed off by one more application of *S3* together with `merge_corollary_regionId_unique_index` (turning "same region" into "same index").

This lemma is where "you can't reach backward in time" stops being a slogan and becomes an actual proof: the reason it takes real work is that the chain is abstract and unboundedly long, so a one-hop fact has to be bootstrapped into an arbitrary-length one by induction, with a genuine case split at the moment a chain crosses from a region onto the stack.

2.4 What "Closed" really buys you

The claim (*CR6*). Once a region is Closed: if nothing outside it already holds its front door (RId), nothing inside it is stack-reachable at all; and if something does hold the front door, stack-reachability of anything inside is exactly region-reachability from the bridge object.

The intuitive story. Closed means sealed. You get in through the front door or not at all, which is close to the entire informal meaning of the word, so this can feel like it should be almost a restatement of the definition.

Why it isn't. *ReachableStep*'s definition only tells you the forward direction: a Closed region's RId has exactly one successor. It says nothing about predecessors, so nothing directly rules out some other, unforeseen way of already being inside. "Sealed" is really a negative claim, that there is no other way in, and proving a negative means closing off every other conceivable entrance: another region's object secretly holding a pointer in (ruled out by *H3*, since a region's own stored references always resolve back into that same region, so nothing external can hold a live pointer into it), and a stack frame that's somehow still executing there (ruled out by *L2*: an on-stack frame can never own a Closed region in the first place).

The argument.

1. `predecessor_of_closed_region_object`: the Closed mirror of §2.3's backward-confinement lemma, except the "escaped to the stack" branch is now impossible (via *L2*'s contrapositive, `closed_region_not_owned`) rather than a base case, and the RId branch, where the predecessor is the region's own portal, becomes available rather than something to rule out.
2. `RegionReachable_of_FrameReachable_closed` / `FrameReachable_rid_of_closed_container`: a backward induction over an arbitrary chain, dispatching at every step on (1)'s two surviving cases: either still tracing inside the region, in which case recurse, or the chain has just used the one portal hop, in which case stop, and note that the remaining tail of the chain, back to its root, already witnesses that the region's own RId was reachable.
3. *CR6* assembles these directly: stack-reachability of anything inside a Closed region factors uniquely through the portal.

The definition of Closed only tells you the one way in that's allowed. *CR6* is the proof that it's also the only way in, and that negative claim needed two separate invariants, *H3* to rule out an outside object holding a pointer in and *L2* to rule out a live stack frame still owning the region, to cover the two entrances the definition alone doesn't.

2.5 A field write in the active frame

The claim (*FRALF* preserved by `fieldAsgn`). If an object living in an earlier (possibly suspended) frame's region was already visible from every later frame before a field write happening in the currently-active frame, it stays visible from that earlier frame afterward.

The intuitive story. A suspended region's contents aren't touched by a field write happening somewhere else entirely. That's the natural first reaction, and it's almost the whole truth, which is what makes it worth being careful about: the natural reaction stops one level too early.

Where intuition needs correcting. The write does create one new edge in the reference graph ($obj.field := newValue$), and nothing about "the write happens somewhere active" rules out, syntactically, that this new edge connects two things that weren't connected before, or shortcuts a longer existing detour. The claim holds not because the new edge is irrelevant by assumption, but because it has to be shown irrelevant: anything already visible from an earlier, strictly-suspended frame is, by §2.3, already confined to have a root no later than that suspended frame itself, which is strictly earlier than where the write happened. So the new edge, whatever it connects, is provably out of reach of any chain that could matter here.

The argument.

1. Let oid_C be the mutated object's id. By §2.3's lemma, now applied to oid_C itself rather than to the tracked object, anything that can reach oid_C at all must come from a frame index no earlier than the active frame's own index, fid_{active}.
2. Any chain confined strictly below fid_{active} never touches oid_C, so the step relation is untouched along it and it transports across the mutation for free. This covers every suspended

frame's chain automatically, since a suspended frame's index is, by hypothesis, strictly below the active one.

3. The remaining case is a chain rooted exactly at the active frame, which is free to have used the new edge. Walk this chain forward hop by hop (`Relation.ReflTransGen.head_induction_on`), asking at each hop whether it's the newly-written one. If so, the chain has jumped onto the field write's new value, which has its own, independent root (`resolveV_frameRoot`), and the argument restarts from there, landing back in the confined case handled by step 2.

The field write's new edge really could matter in general. The proof doesn't get to assume it away; it has to show that only one specific frame's own chain could ever route through it, and even that one case reduces back to an ordinary confined argument once the new edge's endpoint is traced to its own separate root.

2.6 Deleting a region reference

The claim (*FRALF* preserved by `merge`). Same headline property as §2.5, now for the one operation that deletes an entire region, `rid'`, along with the single live reference ($\mathsf{RId}(rid')$) that named it.

The intuitive story. `merge` doesn't create any new objects. It just relabels which heap key some existing objects live under, so reachability from an earlier frame shouldn't change at all.

What that leaves out. A reference that used to exist, $\mathsf{RId}(rid')$, sitting in the merging frame's own variable `x`, is deleted outright and replaced by a reference to `region'.bridgeObjectId`. Any reachability argument that ever routed through the old reference, even implicitly, needs to be re-derived through the new one. "Nothing used it" can't just be asserted here; it has to be shown that nothing could have used it except as a chain's very first step.

The argument.

1. `merge_ridPrime_not_step_target`: $\mathsf{RId}(rid')$ can never be the target of a step. If some reference stepped into it, *H2* (at most one live reference to any region) would already be violated, since `x`'s own occurrence is already the one allowed one. This is a purely counting-based fact, with no case analysis on chains needed.
2. So $\mathsf{RId}(rid')$ can only ever be a chain's root, never mid-chain, and "did this chain use the deleted reference" reduces to one question about the chain's start rather than a hop-by-hop trace.
3. `merge_chain_transport_down`: any chain whose root avoids $\mathsf{RId}(rid')$ transports unconditionally between the two configurations.
4. The one remaining case, a chain rooted exactly at $\mathsf{RId}(rid')$, is handled by `merge_ridPrime_reflTransGen_iff`: since $\mathsf{RId}(rid')$'s only possible first hop is, by definition, its own bridge object, such a chain is the same chain, one hop shorter, rooted at `region'.bridgeObjectId` instead, which is exactly the value `x` now holds post-merge.

`merge` needs its own argument, distinct from an ordinary field write, because it deletes a reference rather than editing one. The proof leans on an existing invariant, *H2*'s one-reference-per-region bound, to show the deleted reference could never have been anything but a chain's starting point, which turns a potentially open-ended re-routing problem into a one-line substitution.

3 Reference: everything else

The rest of `Gc/Model`'s nine operations, the reusable lemma catalog, and known pitfalls. This section is meant for looking things up once you already know which proof you're asking about, rather than for reading start to finish.

3.1 The other operations

`varAsgn x yf` Two branches, on whether `x` is the frame's own bridge variable. If so, this is really a bridge-object reassignment, subtle because it changes *H1*'s witness for that region without touching $objMap$ at all. The proof shows the new bridge id is still a key of the (unchanged) $objMap$, which falls straight out of the rule's own side condition ($yfRef.loc(\cdot) = \mathsf{Rgn}(rid)$ with $rid = frame.regionId$: you can only rebind the bridge to something already inside the same region). If not the bridge var, it's a plain local write gated by $resolveV(\mathtt{x}, \cdot) = \epsilon$ (no existing binding), which exists purely to keep $resolveV$'s outward frame search unambiguous.

`fieldAsgn xf y` The `field-asgn-stack`/`field-asgn-region` split is the clearest illustration of why *H3* needs an explicit $status = Open$ side condition: a region write is permitted only while the writer's frame executes in that exact region and the new value resolves into the same region. Drop either equality and *H3*'s preservation becomes false, since nothing else in the rule would stop the region from acquiring an outward-pointing reference. Its reachability-layer proof is §2.5 above.

`swap x yf` Syntactically the busiest rule, two locations get new values simultaneously, but its invariant proofs are just `fieldAsgn`'s field-write argument and `varAsgn`'s var-write argument run side by side. No new invariant-level content.

`makeObjStack x` / `makeObjRegion x` Allocate a fresh, empty object at $\langle S, H\rangle.freshObjectId$. *L1* preservation is exactly `RuntimeConfig.freshObjectId_not_mem` (`Helpers.lean`): the fresh id is, by construction, not already in $cfg.objectIds$. *H2*/*HS1* for the new id hold vacuously, since nothing points at it yet. `makeObjRegion` additionally needs $status = Open$: an object can only be born directly inside a region while someone is executing there.

`makeRegion x` Allocates a brand-new Closed region with its own fresh bridge object, and stores an RId to it in the current frame. *H2* for the new region id is again vacuous by freshness (`freshRegionId_not_mem`); starting Closed is what keeps *L2* trivially true, since nobody is executing inside the new region yet.

`merge x` See §2.1 (*ValidConfig* preservation) and §2.6 (the reachability-layer proof).

`enter xf bridgeVar` / `exit` Push/pop a frame, flipping the entered/exited region's status. *L2*/*S1* are index bookkeeping; the freshly pushed frame starts with empty $objMap$/$varMap$, so *S2*/*S3*/*HS1* say nothing new about it, since its only content is the just-flipped bridge object, already covered by the region-side invariants. On the reachability side, both are "additive" operations (§3.2): the object graph's content doesn't change, only which frames/regions are active, so the uniform safety-transport tool (`safe_reflTransGen_transport`) applies directly instead of a per-hop induction.

3.2 *FRALF* preservation: the three proof shapes, in one place

§2.5 and §2.6 are two of the three shapes the nine per-operation *FRALF*-preservation proofs take. For completeness, all three:

1. **Additive / status-only** (`enter`, `exit`, `makeObjStack`, `makeObjRegion`, `makeRegion`, `varAsgn`): no existing object's content changes, so `safe_reflTransGen_transport` (§3.3) moves any chain across the mutation with no per-hop case analysis.
2. **Content-mutating** (`fieldAsgn`, `swap`): §2.5's escape argument, keyed on an owner-index bound.

3. **Reference-deleting** (`merge`): §2.6's "can only be a root" argument, keyed on an *H2* count bound.

The corresponding *StackReachable*-invariant proofs (`StackReachableInvariant/`) use the same case analysis per operation, but now prove an $\iff$ rather than a one-way implication. "Was *oid* reachable before" and "is it reachable after" are symmetric questions once confined transport exists in both directions, but the two directions of the escape argument aren't mirror images of each other and have to be proved independently, since the newly-written edge exists in cfg' but not cfg, so the escape case only arises in one of the two directions.

3.3 Reusable lemma catalog

`stackWithIndex_frame_transport_down_of_shape_eq / _up_of_shape_eq` If cfg, cfg' agree up to some projection $\pi : Frame \to \alpha$ at every stack position, $stackWithIndex$ membership transports between them, preserving *.index* and the projected value.

`stackWithIndex_objMap_get_eq_of_last_varMap_update` If only the last frame's $varMap$ changed, $objMap$ is unaffected at every position. Used by every pure var-write operation.

`merge_corollary_regionId_unique_index` *S1* restated at the $FrameWithIndex$ level: two frames sharing a *regionId* are the same frame. Used constantly to turn "same region" facts into "same index" facts.

`swap_corollary_stackWithIndex_index_inj / _find_eq` Two frames with the same *.index* are the same element, since *.index* is assigned by position, not stored data. Probably the single most-cited lemma in `Gc/Reachability/Reachable/`: needed whenever a proof juggles several differently-named frame variables that are provably, but not syntactically, the same frame.

`predecessor_of_region_object / _of_closed_region_object` Backward confinement, one hop, Open/Closed; see §2.3/§2.4.

`SafeRef / predecessor_safe / safe_reflTransGen_transport` A reference is safe for region *rid* if it's in *rid* or on the stack, never a bare RId. Safety propagates backward one hop; if the step relation agrees on every OId-sourced step between cfg, cfg', any chain ending at a safe target transports wholesale. This is the tool behind the "additive" proof shape (§3.2).

`stack_container_confined / region_container_confined` The index-bound lemma of §2.3.

`RegionReachable_of_FrameReachable_closed /FrameReachable_rid_of_closed_container` The Closed-region confinement engine behind *CR6* (§2.4).

The "*H2* forbids a reference from being a step target" trick (§2.6) generalizes beyond `merge`: any future operation that deletes a uniquely-referenced region reference can reuse the same shape (count-based non-membership implies a chain-position restriction, which reduces transport to a root-only case split).

3.4 Pitfalls

Pitfall 3.1 (`AList.insert` reorders on an existing key)**.** Inserting at an already-present key goes through *kinsert*/*kerase* and can reorder *.entries*, unlike inserting at a genuinely fresh key. Any proof needing *.entries*-level equality after an *insert*, not just *.lookup*-level (`merge`'s region-combining argument in §2.1), has to go through an explicit `List.Perm` argument (`exists_of_kerase` + `NodupKeys.eq_of_mk_mem`) rather than assume insert behaves like append.

Pitfall 3.2 (*Status* has no `LawfulBEq`)**.** *Status* derives $BEq/DecidableEq$ but not $LawfulBEq$, so `beq_iff_eq` doesn't fire on it. Every "Open xor Closed" contradiction in this codebase gets closed by case-splitting *region.status* directly and `decide`, not by rewriting a `==` hypothesis.

Pitfall 3.3 (the `<op>_cases` lemma is the semantics)**.** Because the operation is opaque `do`-notation, when in doubt about an operation's actual precondition, read its `<op>_cases` lemma, not the `do`-block. A mistake in the former stays invisible until a downstream theorem quietly stops being provable.

Pitfall 3.4 (Open-only tools misapplied to a Closed region)**.** §2.3's index-confinement lemma (region variant) takes $status = Open$ as a hypothesis, since its proof leans on `predecessor_of_region_object`, which is stated only for the Open case (an object inside a Closed region has no stack-escape branch to bound in the first place). For a Closed region, the right tool is §2.4's confinement lemmas instead. Reaching for the Open-only lemma on a Closed region fails to apply outright rather than silently giving a wrong bound, but it's an easy mismatch when adapting an existing per-operation proof to a new operation.

Pitfall 3.5 ($loc(\cdot)$ treated as opaque)**.** The raw definition of $a.loc(\langle S, H\rangle)$, a nested reverse-stack-search / heap-search with a match forcing exactly one to succeed, is unfolded exactly once, in `Validity.lean`, to prove the three `_iff_in_*` restatement lemmas of §1.2. Every later file treats $.loc(\cdot)$ as an opaque function characterized by those iffs, not by unfolding its definition again.

Pitfall 3.6 (picking the wrong `ReflTransGen` induction principle)**.** Ordinary constructor induction fixes the chain's start and grows its end; `head_induction_on` fixes the end and peels off the start. Using the one that doesn't match what the argument needs to generalize doesn't error out; it produces a stuck goal several tactics later. See §2.2.

Pitfall 3.7 (an $\iff$ needs both halves proved independently)**.** It's tempting to think the $cfg' \to cfg$ direction of an escape argument is just the $cfg \to cfg'$ direction with the arguments flipped. It isn't: the newly-written edge exists in cfg' but not cfg, so the escape case only genuinely arises in one of the two directions and has to be proved on its own terms there. See §3.2.

3.5 What's deliberately not done here

No analogue of a `Guarantees.lean`-style top-level GC-safety theorem sits on top of *StackReachableInvariant* yet. Everything in §2 is a single-step fact; composing it over an arbitrary-length trace, the literal `main.tex`/report.pdf claim, which talks about invariance for as long as a region stays suspended rather than just across one operation, is not currently being pursued.